ITMO University
Faculty of Software Engineering and Computer Systems

Course: Modeling

Research Assignment No. 2 (UIR-2)

"Markov Models of Queueing Systems"

Variant 51/51/30

Student: Rahaman Md Afifur, group P3330

Instructor: _____

Saint Petersburg, 2026

Contents

1 Objective and problem statement **2**

1.1 Input data 2

2 Description of the systems **3**

2.1 SYSTEM_1 3

2.2 SYSTEM_2 3

3 State encoding and list of states **4**

3.1 SYSTEM_1 4

3.2 SYSTEM_2 4

4 Labelled transition graphs **4**

4.1 SYSTEM_1 4

4.2 SYSTEM_2 5

5 Transition rate matrices **6**

5.1 SYSTEM_1 7

5.2 SYSTEM_2 8

6 Stationary state probabilities **8**

7 Performance characteristics **9**

7.1 Verification 10

8 Comparative analysis **11**

8.1 At the given parameters ($\lambda = 0.5$ 1/s, $b = 10$ s) 11

8.2 Dependence on the arrival rate 12

8.3 Effect of request routing in SYSTEM_1 14

9 Choice of the best system **14**

10 Conclusion **15**

Appendix A. Computation program (Python) **15**

1 Objective and problem statement

Objective: to study the method of Markov random processes and apply it to queueing systems with a homogeneous flow of requests.

Task: build Markov models of two alternative queueing systems (SYSTEM_1 and SYSTEM_2), compute the stationary state probabilities and the performance characteristics, compare the systems and choose the better one according to the given efficiency criterion.

1.1 Input data

Assignment variant: $N1/N2/NG = 51/51/30$.

Table 1 – Structural parameters (Table 1 of the assignment)

Variant 1	SYSTEM_1		Variant 2	SYSTEM_2	
	Servers	Buffers		Servers	Buffers
51	2	2/2	51	1 (E_3)	3

Table 2 – Load parameters (Table 2 of the assignment, group 30)

λ, 1/s	b, s	$\mu = 1/b$, 1/s	P1	P2	P3
0.5	10	0.1	0.65	0.2	0.15

Server selection probabilities:

- SYSTEM_1 (two servers): $q_1 = 0.65,\ q_2 = 0.2 + 0.15 = 0.35$;
- SYSTEM_2 (one server): $q = 1$.

Efficiency criterion. Since $N1 + N2 = 51 + 51 = 102$ is even, the criterion is taken from variant $N2 = 51$: **(a) maximum system throughput**.

Total offered load: $Y = \lambda b = 0.5 \cdot 10 = 5$, so both systems are heavily overloaded; a stationary regime still exists because the buffer capacities are finite.

2 Description of the systems

2.1 SYSTEM_1

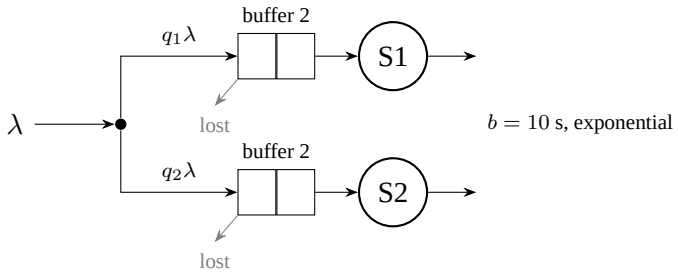


Figure 1 – Structure of SYSTEM_1

- two servers S1 and S2; service time at each is exponential with mean $b = 10$ s ($\mu = 0.1$ 1/s);
- each server has its own buffer of capacity 2 (buffers 2/2);
- the arrival flow is Poisson with rate $\lambda = 0.5$ 1/s; an arriving request goes to S1 with probability $q_1 = 0.65$ ($\lambda_1 = q_1\lambda = 0.325$ 1/s) and to S2 with probability $q_2 = 0.35$ ($\lambda_2 = q_2\lambda = 0.175$ 1/s);
- if the buffer of the chosen server is full, the request is lost (it does not move to the other server);
- queue discipline is FIFO, no priorities.

Splitting a Poisson flow produces independent Poisson flows, so each "buffer + server" subsystem behaves like an M/M/1/3 queue. In this work, however, a single Markov process is built for the whole system.

2.2 SYSTEM_2

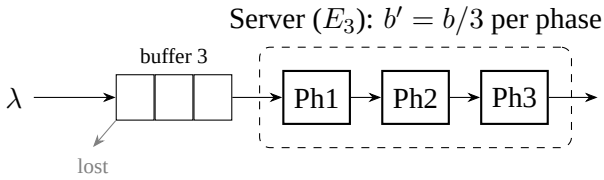


Figure 2 – Structure of SYSTEM_2

- one server; service time follows a 3rd-order Erlang distribution with mean $b = 10$ s (coefficient of variation $\nu = 1/\sqrt{3} \approx 0.577$);
- one shared buffer of capacity 3; at most 4 requests can be in the system;
- the arrival flow is Poisson, $\lambda = 0.5$ 1/s; a request that finds the buffer full is lost.

Reduction to a Markov process. The process in SYSTEM_2 is not Markov because the service time is not exponential. The Erlang-3 distribution is represented as the sum of three exponential phases Ph1, Ph2, Ph3, each with mean $b' = b/3 = 10/3$ s, i.e. phase completion rate $3\mu = 0.3$ 1/s. A process whose state also records the current service phase is then Markov (method of phases).

3 State encoding and list of states

3.1 SYSTEM_1

A state is encoded as a pair $(n_1 n_2)$, where $n_i \in \{0, 1, 2, 3\}$ is the number of requests at server Si (one in service plus up to two in its buffer). The number of states is $4 \cdot 4 = 16 \leq 20$, so no change of the system parameters is needed.

Table 3 – States of SYSTEM_1

State	Code $(n_1 n_2)$	Meaning
E_1	00	system empty
E_2–E_4	01, 02, 03	S1 idle; 1, 2, 3 requests at S2
E_5–E_8	10, 11, 12, 13	S1 serving 1 request, S1 buffer empty
E_9–E_{12}	20, 21, 22, 23	S1 busy, 1 request in S1 buffer
E_{13}–E_{16}	30, 31, 32, 33	S1 busy, S1 buffer full (2 requests)

In general, state $(n_1 n_2)$ has number $E_{4n_1+n_2+1}$; $n_i = 0$ means server Si is idle, $n_i \geq 1$ means it is busy with $n_i - 1$ requests waiting; $n_i = 3$ means its buffer is full and requests routed to Si are lost.

3.2 SYSTEM_2

A state is encoded as 0 (system empty) or n_j, where $n \in \{1, 2, 3, 4\}$ is the total number of requests in the system (one in service, $n - 1$ in the buffer) and $j \in \{1, 2, 3\}$ is the Erlang phase of the request in service. The number of states is $1 + 4 \cdot 3 = 13 \leq 20$.

Table 4 – States of SYSTEM_2

State	Code	Meaning
E_1	0	system empty
E_2, E_3, E_4	$1_1, 1_2, 1_3$	1 request in service (phase 1, 2, 3), buffer empty
E_5, E_6, E_7	$2_1, 2_2, 2_3$	1 in service, 1 in buffer
E_8, E_9, E_{10}	$3_1, 3_2, 3_3$	1 in service, 2 in buffer
E_{11}, E_{12}, E_{13}	$4_1, 4_2, 4_3$	1 in service, 3 in buffer (full)

4 Labelled transition graphs

4.1 SYSTEM_1

Transitions: arrival to S1 ($n_1 \to n_1 + 1$, rate λ_1, if $n_1 < 3$), arrival to S2 ($n_2 \to n_2 + 1$, rate λ_2, if $n_2 < 3$), service completion at S1 or S2 ($n_i \to n_i - 1$, rate μ, if $n_i > 0$).

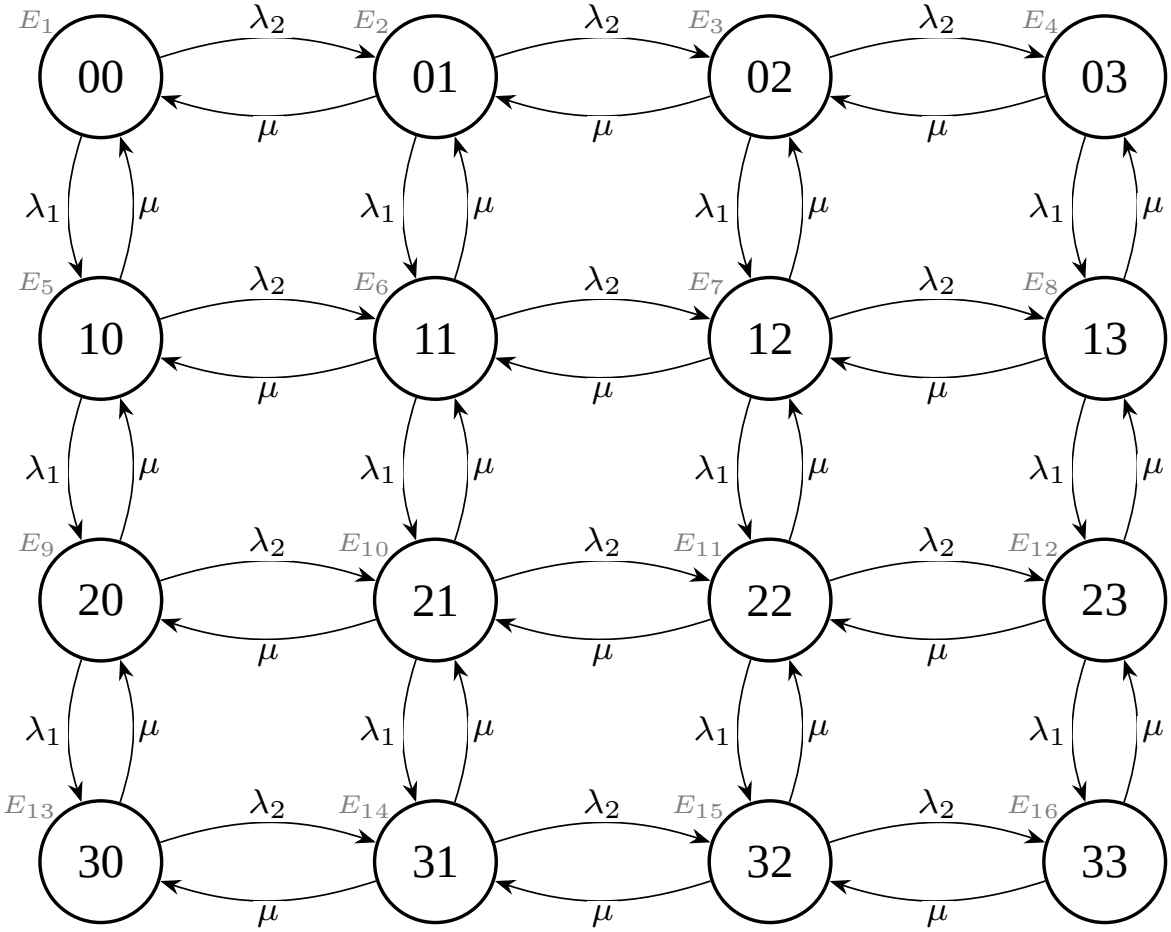


Figure 3 – Labelled transition graph of SYSTEM_1 ($\lambda_1 = 0.325$, $\lambda_2 = 0.175$, $\mu = 0.1$ 1/s); row = n_1, column = n_2

4.2 SYSTEM_2

Transitions: arrival ($n_j \to (n+1)_j$, rate λ, if $n < 4$; from 0 to 1_1), move to the next service phase ($n_j \to n_{j+1}$, rate 3μ), service completion after phase 3 ($n_3 \to (n-1)_1$ or $1_3 \to 0$, rate 3μ); the next request from the buffer immediately starts service in phase 1.

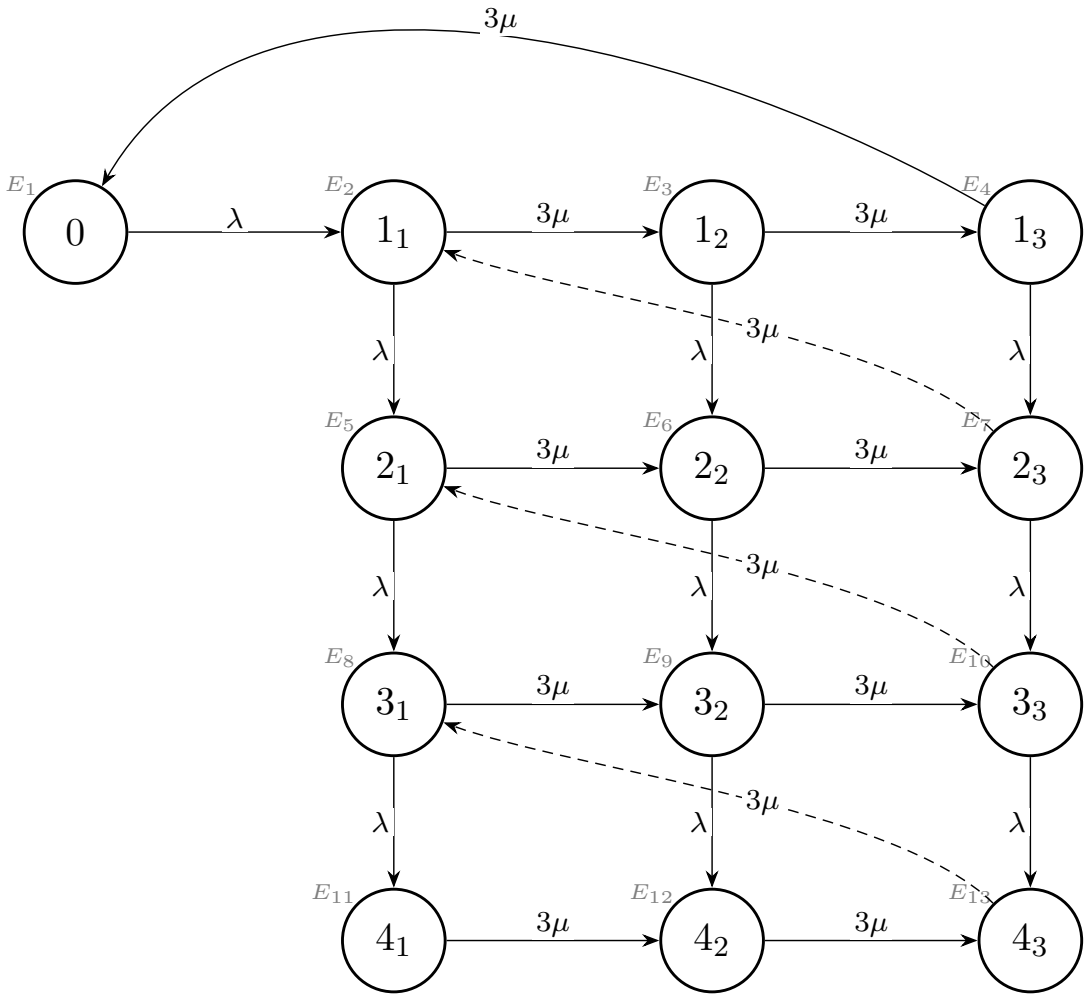


Figure 4 – Labelled transition graph of SYSTEM_2 ($\lambda = 0.5$, $3\mu = 0.3$ 1/s); dashed arcs are service completions

5 Transition rate matrices

Element g_{ij} ($i \neq j$) is the transition rate from E_i to E_j; the diagonal element is $g_{ii} = -\sum_{j \neq i} g_{ij}$ (bold). Empty cells are zeros. The stationary probabilities satisfy

$$\sum_i p_i \, g_{ij} = 0, \quad j = 1, \ldots, n; \qquad \sum_i p_i = 1.$$

5.1 SYSTEM_1

Table 5 – Transition rate matrix G_1 (16×16)

	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}	E_{11}	E_{12}	E_{13}	E_{14}	E_{15}	E_{16}
E_1	**-0.500**	0.175			0.325											
E_2	0.100	**-0.600**	0.175			0.325										
E_3		0.100	**-0.600**	0.175			0.325									
E_4			0.100	**-0.425**				0.325								
E_5	0.100				**-0.600**	0.175			0.325							
E_6		0.100			0.100	**-0.700**	0.175			0.325						
E_7			0.100			0.100	**-0.700**	0.175			0.325					
E_8				0.100			0.100	**-0.525**				0.325				
E_9					0.100				**-0.600**	0.175			0.325			
E_{10}						0.100			0.100	**-0.700**	0.175			0.325		
E_{11}							0.100			0.100	**-0.700**	0.175			0.325	
E_{12}								0.100			0.100	**-0.525**				0.325
E_{13}									0.100				**-0.275**	0.175		
E_{14}										0.100			0.100	**-0.375**	0.175	
E_{15}											0.100			0.100	**-0.375**	0.175
E_{16}												0.100			0.100	**-0.200**

Non-zero rates (in a form convenient for entering into MARK):

$E_1 \to E_2$: $\lambda_2 = 0.175$
$E_1 \to E_5$: $\lambda_1 = 0.325$
$E_2 \to E_1$: $\mu = 0.100$
$E_2 \to E_3$: $\lambda_2 = 0.175$
$E_2 \to E_6$: $\lambda_1 = 0.325$
$E_3 \to E_2$: $\mu = 0.100$
$E_3 \to E_4$: $\lambda_2 = 0.175$
$E_3 \to E_7$: $\lambda_1 = 0.325$
$E_4 \to E_3$: $\mu = 0.100$
$E_4 \to E_8$: $\lambda_1 = 0.325$
$E_5 \to E_1$: $\mu = 0.100$
$E_5 \to E_6$: $\lambda_2 = 0.175$
$E_5 \to E_9$: $\lambda_1 = 0.325$
$E_6 \to E_2$: $\mu = 0.100$
$E_6 \to E_5$: $\mu = 0.100$
$E_6 \to E_7$: $\lambda_2 = 0.175$
$E_6 \to E_{10}$: $\lambda_1 = 0.325$
$E_7 \to E_3$: $\mu = 0.100$
$E_7 \to E_6$: $\mu = 0.100$
$E_7 \to E_8$: $\lambda_2 = 0.175$
$E_7 \to E_{11}$: $\lambda_1 = 0.325$
$E_8 \to E_4$: $\mu = 0.100$
$E_8 \to E_7$: $\mu = 0.100$
$E_8 \to E_{12}$: $\lambda_1 = 0.325$
$E_9 \to E_5$: $\mu = 0.100$
$E_9 \to E_{10}$: $\lambda_2 = 0.175$
$E_9 \to E_{13}$: $\lambda_1 = 0.325$
$E_{10} \to E_6$: $\mu = 0.100$
$E_{10} \to E_9$: $\mu = 0.100$
$E_{10} \to E_{11}$: $\lambda_2 = 0.175$
$E_{10} \to E_{14}$: $\lambda_1 = 0.325$
$E_{11} \to E_7$: $\mu = 0.100$
$E_{11} \to E_{10}$: $\mu = 0.100$
$E_{11} \to E_{12}$: $\lambda_2 = 0.175$
$E_{11} \to E_{15}$: $\lambda_1 = 0.325$
$E_{12} \to E_8$: $\mu = 0.100$
$E_{12} \to E_{11}$: $\mu = 0.100$
$E_{12} \to E_{16}$: $\lambda_1 = 0.325$
$E_{13} \to E_9$: $\mu = 0.100$
$E_{13} \to E_{14}$: $\lambda_2 = 0.175$
$E_{14} \to E_{10}$: $\mu = 0.100$
$E_{14} \to E_{13}$: $\mu = 0.100$
$E_{14} \to E_{15}$: $\lambda_2 = 0.175$
$E_{15} \to E_{11}$: $\mu = 0.100$
$E_{15} \to E_{14}$: $\mu = 0.100$
$E_{15} \to E_{16}$: $\lambda_2 = 0.175$
$E_{16} \to E_{12}$: $\mu = 0.100$
$E_{16} \to E_{15}$: $\mu = 0.100$

5.2 SYSTEM_2

Table 6 – Transition rate matrix G_2 (13×13)

	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}	E_{11}	E_{12}	E_{13}
E_1	**-0.500**	0.500											
E_2		**-0.800**	0.300		0.500								
E_3			**-0.800**	0.300		0.500							
E_4	0.300			**-0.800**			0.500						
E_5					**-0.800**	0.300		0.500					
E_6						**-0.800**	0.300		0.500				
E_7		0.300					**-0.800**			0.500			
E_8								**-0.800**	0.300		0.500		
E_9									**-0.800**	0.300		0.500	
E_{10}					0.300					**-0.800**			0.500
E_{11}											**-0.300**	0.300	
E_{12}												**-0.300**	0.300
E_{13}								0.300					**-0.300**

Non-zero rates:

$E_1 \to E_2$: $\lambda = 0.500$	$E_4 \to E_7$: $\lambda = 0.500$	$E_7 \to E_{10}$: $\lambda = 0.500$	$E_{10} \to E_{13}$: $\lambda = 0.500$
$E_2 \to E_3$: $3\mu = 0.300$	$E_5 \to E_6$: $3\mu = 0.300$	$E_8 \to E_9$: $3\mu = 0.300$	$E_{11} \to E_{12}$: $3\mu = 0.300$
$E_2 \to E_5$: $\lambda = 0.500$	$E_5 \to E_8$: $\lambda = 0.500$	$E_8 \to E_{11}$: $\lambda = 0.500$	$E_{12} \to E_{13}$: $3\mu = 0.300$
$E_3 \to E_4$: $3\mu = 0.300$	$E_6 \to E_7$: $3\mu = 0.300$	$E_9 \to E_{10}$: $3\mu = 0.300$	$E_{13} \to E_8$: $3\mu = 0.300$
$E_3 \to E_6$: $\lambda = 0.500$	$E_6 \to E_9$: $\lambda = 0.500$	$E_9 \to E_{12}$: $\lambda = 0.500$	
$E_4 \to E_1$: $3\mu = 0.300$	$E_7 \to E_2$: $3\mu = 0.300$	$E_{10} \to E_5$: $3\mu = 0.300$	

6 Stationary state probabilities

The stationary probabilities were obtained by solving the linear system of Section 5 (in MARK; as a cross-check the system was also solved numerically in Python/NumPy, see Appendix A). The residual $\max_j |\sum_i p_i g_{ij}|$ is $4.2 \cdot 10^{-17}$ for SYSTEM_1 and $7.4 \cdot 10^{-17}$ for SYSTEM_2; the probabilities sum to 1.

Form 1

Table 7 – Stationary state probabilities

State no.	SYSTEM_1 Code	SYSTEM_1 Prob.	SYSTEM_2 Code	SYSTEM_2 Prob.
1	E_1: $(0,0)$	0.001822	E_1: 0	0.000036
2	E_2: $(0,1)$	0.003188	E_2: 1_1	0.000432
3	E_3: $(0,2)$	0.005578	E_3: 1_2	0.000162
4	E_4: $(0,3)$	0.009762	E_4: 1_3	0.000061
5	E_5: $(1,0)$	0.005920	E_5: 2_1	0.007212
6	E_6: $(1,1)$	0.010360	E_6: 2_2	0.002806
7	E_7: $(1,2)$	0.018130	E_7: 2_3	0.001090
8	E_8: $(1,3)$	0.031727	E_8: 3_1	0.122129
9	E_9: $(2,0)$	0.019240	E_9: 3_2	0.047552
10	E_{10}: $(2,1)$	0.033670	E_{10}: 3_3	0.018513
11	E_{11}: $(2,2)$	0.058922	E_{11}: 4_1	0.203548
12	E_{12}: $(2,3)$	0.103113	E_{12}: 4_2	0.282802
13	E_{13}: $(3,0)$	0.062529	E_{13}: 4_3	0.313657
14	E_{14}: $(3,1)$	0.109426		
15	E_{15}: $(3,2)$	0.191496		
16	E_{16}: $(3,3)$	0.335118		
$\sum$		1.000000		1.000000

The most probable state of SYSTEM_1 is (33), both buffers full ($p = 0.335118$); the probability that the whole system is idle is $p_{00} = 0.001822$. In SYSTEM_2 the probability that the buffer is full ($4_1 + 4_2 + 4_3$) is 0.8000, and the idle probability is $p_0 = 0.000036$. This is expected at a load of $Y = 5$.

7 Performance characteristics

Notation: p is a stationary state probability and each sum runs over the states satisfying the given condition; n_1, n_2 are the numbers of requests at S1 and S2 (SYSTEM_1); n is the number of requests in SYSTEM_2; $(x)^+ = \max(x, 0)$. All characteristics except the times are computed directly from the state probabilities. The loss probability equals the probability that the buffer of the chosen server is full (Poisson arrivals see time averages, PASTA).

Form 2

Table 8 – System characteristics

Characteristic	Server	Formula (Sys. 1)	Formula (Sys. 2)	Sys. 1	Sys. 2
Offered load y	S1	$y_1 = q_1 \lambda b$	$y = \lambda b$	3.2500	5.0000
	S2	$y_2 = q_2 \lambda b$	–	1.7500	–
	Total	$Y = y_1 + y_2$	$Y = y$	5.0000	5.0000
Utilization ρ	S1	$\rho_1 = \sum_{n_1>0} p$	$\rho = 1 - p_0$	0.97965	0.99996
	S2	$\rho_2 = \sum_{n_2>0} p$	–	0.91049	–
	Total	$\rho = (\rho_1 + \rho_2)/2$	ρ	0.94507	0.99996
	Mean busy servers	$R = \rho_1 + \rho_2$	$R = \rho$	1.89014	0.99996
Queue length l	S1	$l_1 = \sum (n_1 - 1)^+ p$	$l = \sum (n-1)^+ p$	1.6121	2.7875
	S2	$l_2 = \sum (n_2 - 1)^+ p$	–	1.2336	–
	Total	$L = l_1 + l_2$	$L = l$	2.8456	2.7875
Requests in system m	S1	$m_1 = \sum n_1 p$	$m = \sum np$	2.5917	3.7875
	S2	$m_2 = \sum n_2 p$	–	2.1441	–
	Total	$M = m_1 + m_2$	$M = m$	4.7358	3.7875
Waiting time w, s	S1	$w_1 = l_1/\lambda'_1$	$w = l/\lambda'$	16.4557	27.8762
	S2	$w_2 = l_2/\lambda'_2$	–	13.5484	–
	Total	$W = L/\Lambda'$	$W = w$	15.0552	27.8762
Time in system u, s	S1	$u_1 = m_1/\lambda'_1$	$u = m/\lambda'$	26.4557	37.8762
	S2	$u_2 = m_2/\lambda'_2$	–	23.5484	–
	Total	$U = M/\Lambda'$	$U = u$	25.0552	37.8762
Loss probability π	S1	$\pi_1 = \sum_{n_1=3} p$	$\pi = \sum_{n=4} p$	0.6986	0.8000
	S2	$\pi_2 = \sum_{n_2=3} p$	–	0.4797	–
	Total	$\pi = 1 - \Lambda'/\lambda$	π	0.6220	0.8000
Throughput λ', 1/s	S1	$\lambda'_1 = q_1 \lambda (1 - \pi_1)$	$\lambda' = \lambda(1 - \pi)$	0.0980	0.1000
	S2	$\lambda'_2 = q_2 \lambda (1 - \pi_2)$	–	0.0910	–
	Total	$\Lambda' = \lambda'_1 + \lambda'_2$	$\Lambda' = \lambda'$	0.1890	0.1000
Loss rate λ'', 1/s	Total	$\lambda'' = \lambda - \Lambda'$	$\lambda'' = \lambda\pi$	0.3110	0.4000

Loss states: for S1 of SYSTEM_1, $n_1 = 3$ (E_{13}–E_{16}); for S2, $n_2 = 3$ (E_4, E_8, E_{12}, E_{16}); for SYSTEM_2, $n = 4$ (E_{11}–E_{13}). The utilization of SYSTEM_1 is given per server ($\rho = R/2$), and R is the mean number of simultaneously busy servers.

7.1 Verification

The results were checked with the fundamental relations:

- server throughput equals $\lambda' = \rho/b$ (output rate of a busy server);
- $m = l + \rho$ (requests in system = waiting + in service);
- Little's law: $u = w + b$;
- for SYSTEM_2 additionally $\lambda' = 3\mu\,(p_{1_3} + p_{2_3} + p_{3_3} + p_{4_3})$;
- for SYSTEM_1: $\pi = q_1\pi_1 + q_2\pi_2$.

Table 9 – Verification of the relations

Server	$\lambda' = \lambda(1-\pi)$	ρ/b	$u = m/\lambda'$	$w + b$	m	$l + \rho$
S1 (Sys1)	0.097965	0.097965	26.4557	26.4557	2.5917	2.5917
S2 (Sys1)	0.091049	0.091049	23.5484	23.5484	2.1441	2.1441
S (Sys2)	0.099996	0.099996	37.8762	37.8762	3.7875	3.7875

All relations hold up to rounding error, which confirms that the models and the computed probabilities are correct.

8 Comparative analysis

8.1 At the given parameters ($\lambda = 0.5$ 1/s, $b = 10$ s)

Table 10 – Comparison of the main characteristics

Characteristic	SYSTEM_1	SYSTEM_2	Better
Throughput Λ', 1/s	0.1890	0.1000	Sys1 (1.89×)
Loss probability π	0.6220	0.8000	Sys1
Server utilization ρ	0.9451	0.99996	Sys2
Mean busy servers R	1.8901	0.99996	Sys1
Total queue length L	2.846	2.788	Sys2 (marginally)
Time in system U, s	25.06	37.88	Sys1

Findings.

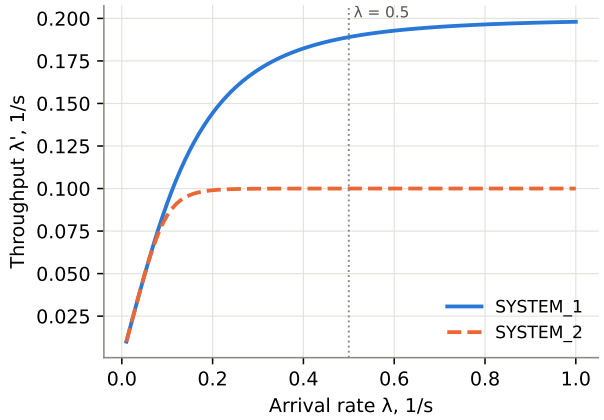
1. The throughput of the single-server SYSTEM_2 is bounded above by $1/b = 0.1$ 1/s, and at load $Y = 5$ this bound is practically reached ($\rho \approx 0.99996$). SYSTEM_1 has two servers, so its upper bound is $2/b = 0.2$ 1/s; its actual throughput of 0.1890 1/s is 1.89 times that of SYSTEM_2.
2. SYSTEM_2 loses 80 % of requests versus 62 % in SYSTEM_1. In SYSTEM_1 most losses come from the more heavily loaded S1 ($\pi_1 = 0.6986$ versus $\pi_2 = 0.4797$).
3. The single server of SYSTEM_2 has higher utilization (it is almost never idle), but this reflects a lack of capacity rather than an advantage: SYSTEM_1 keeps on average 1.8901 servers busy.

4. Total queue lengths are almost equal (2.846 and 2.788), but in SYSTEM_1 the queues are served by two servers in parallel, so the mean time in system is 1.5 times shorter (25.06 s versus 37.88 s).

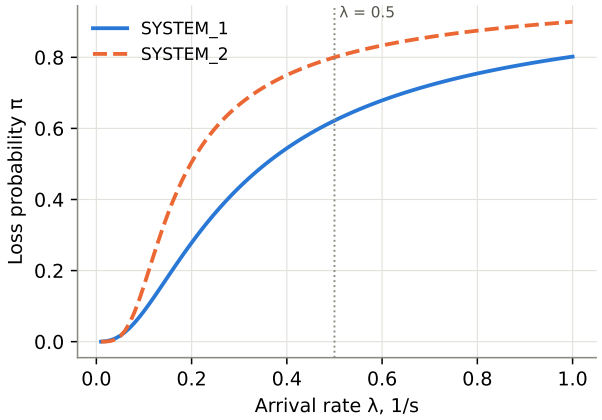
5. Erlang service ($\nu \approx 0.58$) hardly affects the result under such overload: with exponential service SYSTEM_2 would have a throughput of 0.0999 1/s. The deciding factor is the number of servers.

8.2 Dependence on the arrival rate

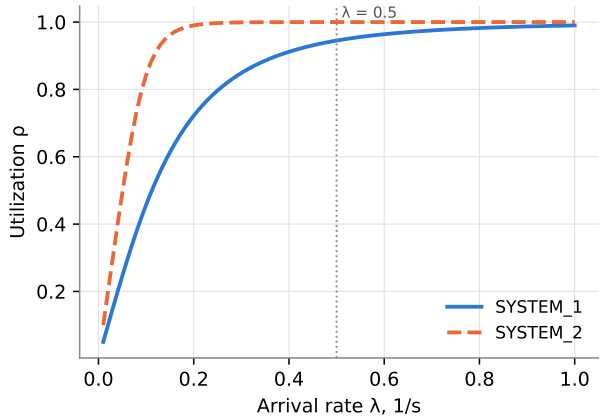
To check how robust the conclusion is, the arrival rate λ was varied from 0.01 to 1 1/s with all other parameters fixed (Fig. 5a–5f).



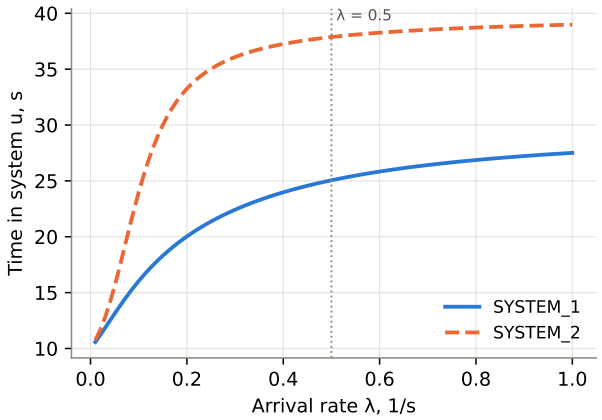
(a) Throughput



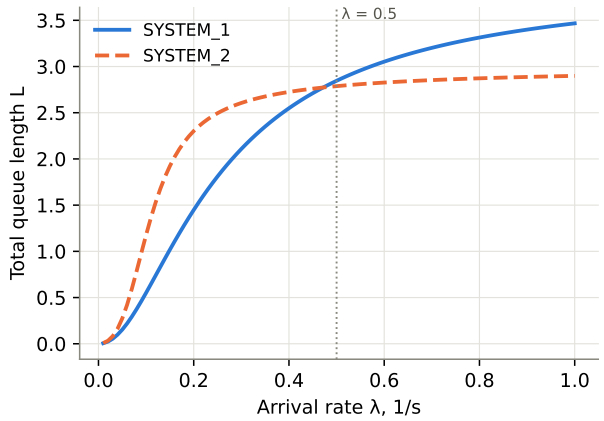
(b) Loss probability



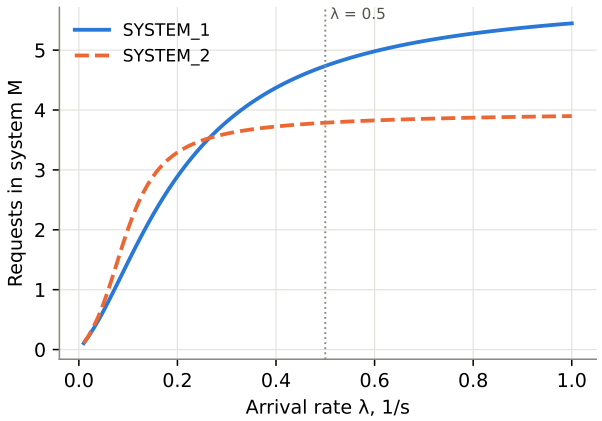
(c) Utilization



(d) Time in system



(e) Total queue length



(f) Requests in system

Figure 5 – System characteristics versus λ (dotted line: the given value $\lambda = 0.5$)

The plots show:

- At low arrival rates ($\lambda \lesssim 0.05$ 1/s) the throughputs of the two systems practically coincide and equal λ, because losses are small. As λ grows, SYSTEM_2 quickly saturates at $1/b = 0.1$ 1/s, while SYSTEM_1 keeps growing towards its limit of 0.2 1/s. Hence $\Lambda'_1 \geq \Lambda'_2$ for every λ.
- SYSTEM_2 has lower losses only for $\lambda < 0.054$ 1/s (there one shared buffer of 3 places is more efficient than two separate buffers of 2); at higher rates SYSTEM_1 loses less.

- The server of SYSTEM_2 is always more utilized: a single server takes the whole flow, whereas in SYSTEM_1 the flow is split between two servers.
- Time in system is shorter in SYSTEM_1 for every λ, and the gap widens with load.
- The total queue length and the number of requests in SYSTEM_1 exceed those of SYSTEM_2 for $\lambda > 0.47$ and $\lambda > 0.26$ 1/s respectively, because SYSTEM_1 has a larger total capacity (2 servers + 4 places versus 1 server + 3 places). At the given $\lambda = 0.5$ this shows only as a slightly larger L.

8.3 Effect of request routing in SYSTEM_1

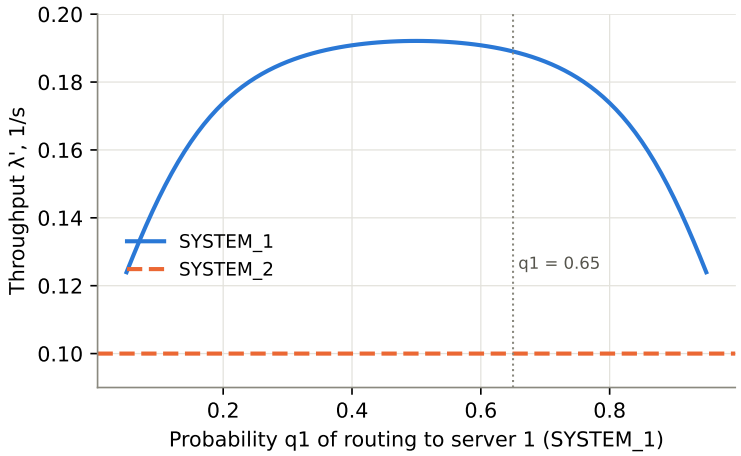


Figure 6 – Throughput of SYSTEM_1 versus routing probability q_1

SYSTEM_1 reaches its maximum throughput with an even split ($q_1 = 0.5$): 0.1921 1/s. At the given $q_1 = 0.65$ it is only $\approx 1.6\,\%$ lower, and for every $q_1 \in [0.05,\ 0.95]$ SYSTEM_1 stays ahead of SYSTEM_2.

9 Choice of the best system

The efficiency criterion is **maximum system throughput** (the rate of the flow of served requests, Λ'):

$$\Lambda'_1 = 0.1890 \text{ 1/s} \ > \ \Lambda'_2 = 0.1000 \text{ 1/s}.$$

The best option is SYSTEM_1 (two exponential servers with separate buffers of capacity 2/2): its throughput is 1.89 times higher. SYSTEM_1 is also better in loss probability and time in system; SYSTEM_2 wins only on single-server utilization and, marginally, on total queue length, neither of which is the given criterion. The conclusion holds for any arrival rate: at small λ the throughputs coincide, and as λ grows the advantage of SYSTEM_1 increases.

10 Conclusion

Markov models were built for two queueing systems: a two-server system with separate buffers (16 states) and a single-server system with 3rd-order Erlang service (13 states, method of phases). For each system the labelled transition graph and the transition rate matrix were constructed, the stationary probabilities were computed, and the full set of characteristics was derived from the state probabilities; correctness was confirmed with the fundamental relations (Little's law and others). By the maximum-throughput criterion, SYSTEM_1 was chosen.

Appendix A. Computation program (Python)

```
"""UIR-2, variant 51/51/30: Markov models of two queueing systems.
SYSTEM_1: 2 exponential servers, separate buffers 2/2, routing q1=0.65, q2=0.35.
SYSTEM_2: 1 server with Erlang-3 service, buffer 3.
Load (group 30): lambda=0.5 1/s, b=10 s.
"""
import numpy as np

LAM, B = 0.5, 10.0
Q1, Q2 = 0.65, 0.35 # P2 = 0.2 + 0.15 for a two-server system
K1 = K2 = 2 # buffer sizes of SYSTEM_1
K = 3 # shared buffer of SYSTEM_2
ERL = 3 # Erlang order

def stationary(G):
   """Solve p G = 0, sum p = 1 (G with diagonal = -row sums)."""
   n = G.shape[0]
   A = G.T.copy()
   A[-1, :] = 1.0
   rhs = np.zeros(n); rhs[-1] = 1.0
   return np.linalg.solve(A, rhs)

def finalize(G):
   np.fill_diagonal(G, 0.0)
   np.fill_diagonal(G, -G.sum(axis=1))
   return G

# ---------------- SYSTEM_1 ----------------
def sys1_states(k1=K1, k2=K2):
   return [(a, c) for a in range(k1 + 2) for c in range(k2 + 2)]

def sys1(lam=LAM, b=B, q1=Q1, k1=K1, k2=K2):
   mu = 1 / b; l1, l2 = q1 * lam, (1 - q1) * lam
   S = sys1_states(k1, k2); idx = {s: i for i, s in enumerate(S)}
   G = np.zeros((len(S), len(S)))
   sym = [["" for _ in S] for _ in S]
   for (a, c), i in idx.items():
      if a < k1 + 1: G[i, idx[(a + 1, c)]] += l1; sym[i][idx[(a + 1, c)]] = r"\lambda_1"
      if c < k2 + 1: G[i, idx[(a, c + 1)]] += l2; sym[i][idx[(a, c + 1)]] = r"\lambda_2"
      if a > 0: G[i, idx[(a - 1, c)]] += mu; sym[i][idx[(a - 1, c)]] = r"\mu"
      if c > 0: G[i, idx[(a, c - 1)]] += mu; sym[i][idx[(a, c - 1)]] = r"\mu"
   G = finalize(G); p = stationary(G)
```

```
    P = dict(zip(S, p))
    r = {}
    r["y"] = [l1 * b, l2 * b]
    r["rho"] = [sum(v for (a, c), v in P.items() if a > 0), sum(v for (a, c), v in P.items
        () if c > 0)]
    r["l"] = [sum(max(a - 1, 0) * v for (a, c), v in P.items()), sum(max(c - 1, 0) * v for
         (a, c), v in P.items())]
    r["m"] = [sum(a * v for (a, c), v in P.items()), sum(c * v for (a, c), v in P.items())
        ]
    r["pi"] = [sum(v for (a, c), v in P.items() if a == k1 + 1), sum(v for (a, c), v in P.
        items() if c == k2 + 1)]
    r["thr"] = [l1 * (1 - r["pi"][0]), l2 * (1 - r["pi"][1])]
    r["w"] = [r["l"][i] / r["thr"][i] for i in range(2)]
    r["u"] = [r["m"][i] / r["thr"][i] for i in range(2)]
    T = {}
    T["y"] = sum(r["y"]); T["rho"] = sum(r["rho"]) / 2; T["R"] = sum(r["rho"])
    T["l"] = sum(r["l"]); T["m"] = sum(r["m"]); T["thr"] = sum(r["thr"])
    T["pi"] = 1 - T["thr"] / lam
    T["w"] = T["l"] / T["thr"]; T["u"] = T["m"] / T["thr"]
    # checks (Little etc.)
    chk = {
        "thr_vs_rho/b": [(r["thr"][i], r["rho"][i] / b) for i in range(2)],
        "m=l+rho": [(r["m"][i], r["l"][i] + r["rho"][i]) for i in range(2)],
        "u=w+b": [(r["u"][i], r["w"][i] + b) for i in range(2)],
        "pi_total=q1pi1+q2pi2": (T["pi"], q1 * r["pi"][0] + (1 - q1) * r["pi"][1]),
    }
    return dict(S=S, G=G, sym=sym, p=p, per=r, tot=T, chk=chk)

# ---------------- SYSTEM_2 ----------------
def sys2_states(k=K, e=ERL):
    return [(0, 0)] + [(n, j) for n in range(1, k + 2) for j in range(1, e + 1)]

def sys2(lam=LAM, b=B, k=K, e=ERL):
    nu = e / b # phase rate
    S = sys2_states(k, e); idx = {s: i for i, s in enumerate(S)}
    G = np.zeros((len(S), len(S)))
    sym = [["" for _ in S] for _ in S]
    for (n, j), i in idx.items():
        if n == 0:
            G[i, idx[(1, 1)]] += lam; sym[i][idx[(1, 1)]] = r"\lambda"; continue
        if n < k + 1: G[i, idx[(n + 1, j)]] += lam; sym[i][idx[(n + 1, j)]] = r"\lambda"
        if j < e: G[i, idx[(n, j + 1)]] += nu; sym[i][idx[(n, j + 1)]] = r"%d\mu" % e
        else:
            t = (0, 0) if n == 1 else (n - 1, 1)
            G[i, idx[t]] += nu; sym[i][idx[t]] = r"%d\mu" % e
    G = finalize(G); p = stationary(G)
    P = dict(zip(S, p))
    r = {}
    r["y"] = lam * b
    r["rho"] = 1 - P[(0, 0)]; r["R"] = r["rho"]
    r["l"] = sum(max(n - 1, 0) * v for (n, j), v in P.items())
    r["m"] = sum(n * v for (n, j), v in P.items())
    r["pi"] = sum(v for (n, j), v in P.items() if n == k + 1)
    r["thr"] = lam * (1 - r["pi"])
    r["w"] = r["l"] / r["thr"]; r["u"] = r["m"] / r["thr"]
```

```
    chk = {
        "thr_vs_rho/b": (r["thr"], r["rho"] / b),
        "thr_vs_phase3": (r["thr"], nu * sum(v for (n, j), v in P.items() if j == e)),
        "m=l+rho": (r["m"], r["l"] + r["rho"]),
        "u=w+b": (r["u"], r["w"] + b),
    }
    return dict(S=S, G=G, sym=sym, p=p, per=r, tot=r, chk=chk)

if __name__ == "__main__":
    np.set_printoptions(linewidth=200, precision=4, suppress=True)
    s1, s2 = sys1(), sys2()
    for name, s in (("SYSTEM_1", s1), ("SYSTEM_2", s2)):
        print("=" * 20, name, len(s["S"]), "states")
        for st, v in zip(s["S"], s["p"]): print(st, f"{v:.6f}")
        print("sum p =", s["p"].sum(), " max|pG| =", np.abs(s["p"] @ s["G"]).max())
        print("per:", s["per"]); print("tot:", s["tot"]); print("chk:", s["chk"])
    print("SYSTEM_2 with exponential service (E1):", sys2(e=1)["tot"])
```